

Gesamtdokumentation WebLab

Architektur, Fazit & Reflexion sowie Arbeitsjournal

Autor(en):	Vorname Nachname (vorname.nachname@stud.hslu.ch) – Matrikelnr.: XX-XXX-XXX
Modul:	WebLab (FS 2026)
Dozierende/Betreuung:	Dozent Name
Version:	1.0
Datum:	31.08.2026

Management Summary / Abstract

Dieses Gesamtdokument beinhaltet alle drei obligatorischen Teildokumente des Moduls WebLab: Die vollständige Architekturdokumentation inklusive ADRs, das abschliessende Fazit mit persönlicher und technischer Reflexion sowie das lückenlose Arbeitsjournal.

Inhaltsverzeichnis

1	TEIL I: Architekturdokumentation	3
2	Einführung & Ziele	3
2.1	Aufgabenstellung & Motivation	3
2.2	Qualitätsziele	3
2.3	Stakeholder & Zielgruppen	3
3	Randbedingungen & Technologiestack	4
3.1	Technische Rahmenbedingungen	4
3.2	Organisatorische & rechtliche Rahmenbedingungen	7
3.3	Konventionen & Richtlinien	7
4	Kontextabgrenzung	8
4.1	Fachlicher Kontext	8
4.2	Technischer Kontext	8
5	Bausteinsicht	9
5.1	Gesamtsystem (Ebene 1)	9
5.2	Frontend-Bausteine (Ebene 2)	9
5.3	Backend-Bausteine (Ebene 2)	9
6	Laufzeitsicht	9
6.1	Typischer Anwendungsfall: Benutzer-Authentifizierung	9
6.2	Datenabruf und State-Update	10
7	Verteilungssicht & Deployment	10
7.1	Infrastruktur-Übersicht	10
7.2	CI/CD-Pipeline	10
8	Datenmodell & Persistenz	11
8.1	Entitäten & Relationen	11
8.2	Datenmigration & Validierung	11
9	Querschnittskonzepte	11
9.1	Sicherheitskonzept (Security)	11
9.2	Fehlerbehandlung & Logging	12
9.3	UI/UX & Responsive Design	12
10	Architekturentscheidungen (ADRs)	12
10.1	Übersicht der Architekturentscheidungen	12
11	TEIL II: Fazit & Reflexion	14
12	Zusammenfassung des Projekts	14
12.1	Projektüberblick & Ausgangslage	14
12.2	Wichtigste Projektergebnisse	14
13	Zielerreichung & Soll-Ist-Vergleich	14
13.1	Funktionaler Soll-Ist-Vergleich	14
13.2	Nicht-funktionale Anforderungen	15
14	Erkenntnisse & Lessons Learned	15
14.1	Technische Erkenntnisse	15
14.2	Aufgetretene Herausforderungen & Lösungen	15
15	Persönliche & Methodische Reflexion	15
15.1	Vorgehensweise & Arbeitsmethodik	15
15.2	Kompetenzgewinn	16

16	Ausblick & Weiterentwicklung	16
16.1	Potenzielle Erweiterungen	16
16.2	Fazit	16
17	TEIL III: Arbeitsjournal	17
18	Übersicht & Zeitbudget	17
18.1	Zielsetzung des Arbeitsjournals	17
18.2	Zeitbudget-Übersicht (Soll / Ist)	17
19	Chronologische Journaleinträge	17
20	Aufwandsanalyse & Fazit zum Arbeitsprozess	19
20.1	Verteilung des Aufwands	19
20.2	Erkenntnisse aus dem Arbeitsprozess	19

1 TEIL I: Architekturdokumentation

2 Einführung & Ziele

2.1 Aufgabenstellung & Motivation

TODO

Beschreiben Sie hier kurz die Problemstellung und die Vision des WebLab-Projekts. Was ist das Hauptziel der Webapplikation?

Die vorliegende Webanwendung wurde im Rahmen des Moduls **WebLab** an der Hochschule Luzern (Departement Informatik) entwickelt. Ziel ist es, eine modulare, performante und benutzerfreundliche Webanwendung zu realisieren, welche moderne Best Practices im Fullstack-Bereich demonstriert.

2.2 Qualitätsziele

Die zentralen Qualitätsziele orientieren sich an ISO/IEC 25010 und sind in folgender Priorität definiert:

Priorität	Qualitätsmerkmal	Konkretisierung / Messbarkeit
1	Benutzerfreundlichkeit (Usability)	Intuitive Bedienung, konsistentes UI/UX, Reaktionszeit bei Interaktionen < 100ms.
2	Wartbarkeit (Maintainability)	Saubere modulare Trennung (Clean Architecture / Komponenten), TypeScript Typisierung, Testabdeckung.
3	Zuverlässigkeit (Reliability)	Robuste Fehlerbehandlung im Frontend & Backend, Valide Datenvalidierung auf API-Ebene.
4	Performance	Optimierte Asset-Bundles, schnelles Initial Rendering (< 1.5s First Contentful Paint).
5	Sicherheit (Security)	Sichere Authentifizierung, Schutz vor XSS/CSRF, Input-Sanitizing.

2.3 Stakeholder & Zielgruppen

Rolle / Stakeholder	Erwartungshaltung	Be- deu- tung für das Sys- tem
Endbenutzer	Schnelle, selbsterklärende und fehlerfreie Nutzung der Kernfunktionen.	Di- rek-

Rolle / Stakeholder	Erwartungshaltung	Be- deu- tung für das Sys- tem
		te In- ter- ak- ti- on mit dem Front- end.
Dozierende / Experten	Technisch saubere Umsetzung, Einhaltung der WebLab-Vorgaben, Dokumentation.	Be- wer- tung & Ab- nah- me.
Entwickler / Betreiber	Gute Erweiterbarkeit, klare Code-Struktur, einfaches lokales Setup und Deployment.	War- tung und Wei- ter- ent- wick- lung.

3 Randbedingungen & Technologiestack

3.1 Technische Rahmenbedingungen

Die Entwicklung basiert auf modernen Webtechnologien und unterliegt folgenden Rahmenbedingungen:

Bereich	Technologie / Standard	Be-gründung / Kon-text
Frontend	TypeScript, Modernes Framework (z. B. Angular / React / Vue)	Mo-der-ne Kom-po-nen-ten-ar-chi-tek-tur, Typ-sicher-heit und Sta-te-Ma-nage-ment.
Styling	CSS3 / SCSS / Tailwind CSS	Re-spon-si-ve De-sign, Kon-sis-ten-tes De-sign Sys-tem.
Backend / API	Node.js / Express / NestJS / REST API	Asyn-chron-es I/O, naht-

Bereich	Technologie / Standard	Be- grün- dung / Kon- text
		lo- se Type- Script- In- tegra- ti- on mit Shared Ty- pes.
Datenbank / Storage	PostgreSQL / MongoDB / SQLite	Struk- turier- te Da- ten- spei- che- rung und re- latio- na- le bzw. do- ku- men- ten- ba- sier- te In- tegri- tät.
Versionskontrolle	Git & GitHub	Fea- ture- Branch- Work-

Bereich	Technologie / Standard	Be-gründung / Kon-text
		flow, Pull Requests und Code Reviews.
Dokumentation	Typst	Präzise, versionierbare und moderne PDF-Generierung.

3.2 Organisatorische & rechtliche Rahmenbedingungen

- **Lizenzen:** Einsatz ausschliesslich von Open-Source-Bibliotheken unter permissiven Lizenzen (MIT, Apache 2.0).
- **Datenschutz:** Es werden keine sensiblen Personendaten ohne explizite Einwilligung oder im Produktivbetrieb unverschlüsselt verarbeitet (DSG / DSGVO Konformität im Konzept).
- **Abgabetermin & Meilensteine:** Einhaltung der vorgegebenen Modul-Deadlines gemäss WebLab-Zeitplan.

3.3 Konventionen & Richtlinien

- **Code Style:** ESLint & Prettier für einheitliche Code-Formatierung.
- **Naming Conventions:** CamelCase für Variablen/Funktionen, PascalCase für Klassen/Komponenten, kebab-case für Dateinamen.
- **Git Commits:** Conventional Commits (`feat:` , `fix:` , `docs:` , `refactor:` , `test:` , `chore:`).

4 Kontextabgrenzung

4.1 Fachlicher Kontext

Der fachliche Kontext definiert die Systemgrenze aus Benutzersicht und beschreibt die externen Akteure sowie den Informationsaustausch.

i Hinweis

Fügen Sie bei Bedarf ein Kontextdiagramm (z. B. via Mermaid oder exportiertes Bild aus Doc/assets/) ein.

Nachbarsystem / Akteur	Eingabe in das System	Ausgabe aus dem System
Benutzer (Webbrowser)	Benutzerinteraktionen, Formulareingaben, Authentifizierungsdaten	Dynamische UI-Views, Bestätigungen, Fehlermeldungen
Dritte APIs (z. B. OAuth, Wetter, etc.)	API-Tokens, externe Daten-Payloads	Anfragen, Webhook-Trigger

4.2 Technischer Kontext

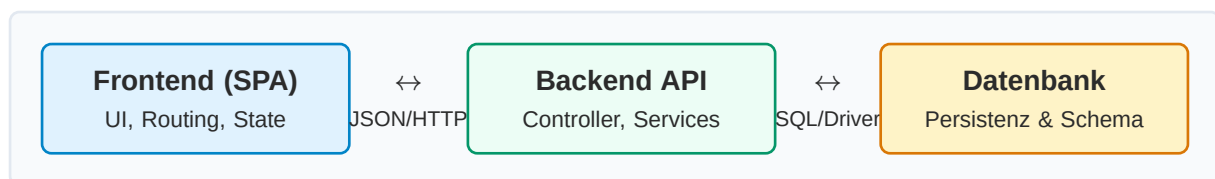
Der technische Kontext beschreibt die Kommunikationskanäle und Protokolle zwischen den Komponenten:

- **Frontend** ⇒ **Backend**: HTTP/HTTPS via RESTful JSON API (bzw. WebSockets für Echtzeitkommunikation).
- **Backend** ⇒ **Datenbank**: TCP/IP über standardisierte Datenbanktreiber / ORM mit Connection-Pooling.
- **Client-Plattformen**: Moderne Desktop- und Mobil-Browser (Chromium, Firefox, Safari) mit responsivem Viewport.

5 Bausteinsicht

5.1 Gesamtsystem (Ebene 1)

Das Gesamtsystem gliedert sich in eine klassische 3-Schichten-Architektur:



5.2 Frontend-Bausteine (Ebene 2)

Das Frontend ist nach Feature-Modulen und Shared-Komponenten modularisiert:

- **Core / App-Shell**: Grundlegendes Layout, Header, Navigation, globale Interceptoren und State-Services.
- **Feature-Module**: Kapselung von fachlichen Domänen (z. B. Authentifizierung, Dashboard, Datenansicht, Einstellungen).
- **Shared / UI-Components**: Wiederverwendbare atomare UI-Elemente (Buttons, Modals, Form Controls, Tables).
- **Services & API-Clients**: Datenabruf, Caching und Synchronisation mit dem Backend.

5.3 Backend-Bausteine (Ebene 2)

Das Backend folgt dem Schichtenmodell zur Trennung von Belangen (Separation of Concerns):

1. **Controller / Routing-Layer**: Entgegennahme von HTTP-Requests, Validierung der Payloads (DTOs) und HTTP-Response-Handling.
2. **Service- / Business-Logic-Layer**: Zentrale Geschäftslogik, Domain-Validierung und Orchestrierung.
3. **Data Access / Repository-Layer**: Abstraktion des Datenbankzugriffs, CRUD-Operationen und Transaktionsverwaltung.
4. **Middleware**: Authentifizierung (JWT Verification), Logging, CORS und globales Error-Handling.

6 Laufzeitsicht

6.1 Typischer Anwendungsfall: Benutzer-Authentifizierung

Der Ablauf für Login und Autorisierung gestaltet sich wie folgt:

1. Der Benutzer gibt Zugangsdaten im Anmeldeformular ein.
2. Das Frontend validiert das Formular lokal und sendet einen `POST /api/auth/login` Request an das Backend.
3. Der Backend-Auth-Controller validiert die Eingaben, hasht das Passwort und vergleicht den Hash mit dem Datenbankeintrag.

- Bei erfolgreicher Verifikation generiert das Backend ein signiertes JSON Web Token (JWT) und gibt dieses zurück.
- Das Frontend speichert den Token sicher (z. B. Secure HTTP-Only Cookie oder im Auth-State) und fügt ihn bei Folgeanfragen im `Authorization: Bearer <token>` Header ein.

6.2 Datenabruf und State-Update

Schritt	Beteiligte Komponente	Aktion / Beschreibung
1	UI-Komponente	Löst beim Rendern oder durch Benutzeraktion ein Event / Dispatch aus.
2	Data Service	Sendet einen HTTP GET Request an den zuständigen API-Endpunkt.
3	Backend API	Verifiziert den JWT-Token, prüft Berechtigungen und holt Daten via ORM/SQL.
4	Frontend Store	Aktualisiert den lokalen State mit der erhaltenen Response.
5	UI-Komponente	Reagiert reaktiv auf die Zustandsänderung und aktualisiert die Ansicht.

✓ Tipp

Sequenzdiagramme können direkt in Typst oder als Vektorgrafiken (`.svg`) in `Doc/assets/` abgelegt und eingebunden werden.

7 Verteilungssicht & Deployment

7.1 Infrastruktur-Übersicht

Die Webanwendung ist für containerisierte Umgebungen konzipiert und kann sowohl lokal als auch in Cloud-Infrastrukturen betrieben werden.

Knoten / Container	Umgebung	Beschreibung / Konfiguration
Frontend Static Host	Nginx / Cloudflare Pages / Vercel	Auslieferung kompilierter Produktions-Assets (HTML, JS, CSS, Media).
Backend Service	Node.js Container (Docker)	Containerisierter Backend-Prozess, REST API, Environment-Konfiguration via <code>.env</code> .
Database Instance	PostgreSQL / MongoDB	Persistentes Volume, automatische Migrationen bei Start.

7.2 CI/CD-Pipeline

Zur kontinuierlichen Qualitätssicherung ist eine automatisierte Pipeline (z. B. via GitHub Actions) vorgesehen:

1. **Linting & Type-Check:** Automatische Überprüfung via `npm run lint` und `tsc --noEmit`.
2. **Automatisierte Tests:** Ausführung von Unit- und Integrationstests (`npm test`).
3. **Build-Prozess:** Kompilierung des Frontends (`npm run build`) und Backends.
4. **Container Build & Registry:** Erstellung der Docker-Images und Tagging mit Git-Commit-Hash.

8 Datenmodell & Persistenz

8.1 Entitäten & Relationen

Das Datenmodell bildet die fachlichen Kernentitäten der Anwendung ab.



TODO

Passen Sie die nachfolgende Entitäten-Tabelle an Ihr konkretes Datenbankschema an.

Entität	Feld	Typ	Beschreibung / Constraints
User	id	UUID / Int	Primary Key, automatische Generierung
	email	String	Unique, Pflichtfeld, validierte E-Mail-Adresse
	passwordHash	String	Bcrypt- / Argon2-Hash, nicht exponiert
	role	Enum	Rolle (ADMIN , USER)
	createdAt	Timestamp	Erstellungszeitpunkt
EntityItem	id	UUID / Int	Primary Key
	userId	UUID / Int	Foreign Key → User(id)
	title	String(255)	Titel / Bezeichnung
	status	Enum	Status (DRAFT , PUBLISHED , ARCHIVED)
	updatedAt	Timestamp	Letztes Änderungsdatum

8.2 Datenmigration & Validierung

- **Schema-Migrationen:** Werden über ORM-Migrationstools (z. B. Prisma, TypeORM, Drizzle) versioniert und deterministisch ausgeführt.
- **Validierung:** Doppelte Validierung sowohl im Frontend (für unmittelbares Benutzerfeedback) als auch zwingend im Backend (via Schema-Validierung wie Zod / class-validator).

9 Querschnittskonzepte

9.1 Sicherheitskonzept (Security)

- **Authentifizierung & Autorisierung:** Token-basierte Authentifizierung (JWT) mit angemessener Expiration Time und Role-Based Access Control (RBAC).
- **Schutz gegen gängige Web-Schwachstellen (OWASP Top 10):**
 - **XSS:** Automatisches Escaping durch das Frontend-Framework, Content Security Policy (CSP).
 - **CSRF:** SameSite-Cookie-Flags und CSRF-Tokens für zustandsbehaftete Mutationen.

- **SQL-Injection:** Ausschliessliche Verwendung von parametrisierten Queries über ORM / Query Builder.
- **HTTPS & Verschlüsselung:** Durchgehende TLS-Verschlüsselung im Transit, Passwort-Hashing mit Argon2id / Bcrypt.

9.2 Fehlerbehandlung & Logging

- **Frontend:** Globaler Error-Boundary-Handler, benutzerfreundliche Toast-Nachrichten, Graceful Degradation bei Verbindungsunterbrüchen.
- **Backend:** Zentraler Exception-Filter / Middleware, strukturierte JSON-Logs (z. B. Pino / Winston) mit Request-IDs zur Nachverfolgbarkeit.
- **API-Fehlerformat:** Standardisierte RFC 7807 (Problem Details for HTTP APIs) Fehlerrückgaben:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Validierungsfehler im Feld 'email'",
  "timestamp": "2026-03-01T10:00:00.000Z"
}
```

9.3 UI/UX & Responsive Design

- Mobile-First-Ansatz mit Breakpoints für Mobile (< 640px), Tablet (640 – 1024px) und Desktop (> 1024px).
- Barrierefreiheit (a11y) gemäss WCAG 2.1 AA Kriterien (Farbkontraste, Tastaturbedienbarkeit, ARIA-Labels).

10 Architekturentscheidungen (ADRs)

In diesem Kapitel werden fundamentale Entwurfs- und Architekturentscheidungen dokumentiert und begründet.

10.1 Übersicht der Architekturentscheidungen

ADR-001: Wahl des Frontend-Frameworks und TypeScript Akzeptiert 15.02.2026

Kontext & Problemstellung:

Für die Entwicklung der WebLab-Anwendung wird ein modernes Frontend-Framework benötigt, welches strikte Typsicherheit, komponentenbasierte Wiederverwendbarkeit und ein starkes Ökosystem bietet.

Entscheidung:

Wir setzen auf ein modernes Single-Page-Application-Framework mit durchgängigem Einsatz von TypeScript.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Hohe Wartbarkeit, statische Fehlerprüfung zur Compile-Zeit, hervorragende IDE-Unterstützung und Autovervollständigung.
- **Nachteile / Trade-offs:** Geringfügiger Boilerplate-Overhead beim Erstellen von Interfaces und DTOs.

ADR-002: Authentifizierung via JWT (JSON Web Tokens) Akzeptiert 20.02.2026

Kontext & Problemstellung:

Das System erfordert eine sichere, zustandslose Authentifizierung zwischen Frontend-SPA und Backend-API.

Entscheidung:

Einsatz von JSON Web Tokens (JWT) mit HMAC-SHA256 Signatur und kurzer Gültigkeitsdauer.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Stateless Backend (keine Server-Sessions in der DB erforderlich), gut skalierbar, einfache Handhabung im REST-Kontext.
- **Nachteile / Trade-offs:** Token-Revocation (Blacklisting) erfordert zusätzliche Mechanismen wie Refresh Tokens oder Redis-Cache.

ADR-003: Dokumentationserstellung mit Typst Akzeptiert 25.02.2026

Kontext & Problemstellung:

Die geforderten Projektdokumente (Architektur, Reflexion, Journal) müssen im Team sauber versioniert, ansprechend formatiert und schnell als PDF exportiert werden können.

Entscheidung:

Nutzung von Typst anstelle von Word oder schwergewichtigem LaTeX.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Sehr schnelle Kompilierung (< 100ms), saubere Code-basierte Struktur in Git, moderne Syntax, hervorragendes Typography-Layout.
- **Nachteile / Trade-offs:** Einarbeitung in Typst-Syntax für Teammitglieder nötig (wird durch Vorlagen stark vereinfacht).

11 TEIL II: Fazit & Reflexion

12 Zusammenfassung des Projekts

12.1 Projektüberblick & Ausgangslage

TODO

Fassen Sie hier das Projekt in wenigen Absätzen zusammen: Was wurde gebaut, wer waren die Hauptnutzer und wie verlief die Entwicklung?

Im Modul WebLab wurde eine vollwertige Webapplikation konzipiert, architektonisch aufgebaut und implementiert. Das Hauptaugenmerk lag auf der Beherrschung des modernen Web-Stacks, der sauberen Trennung von Frontend, Backend und Persistenzschicht sowie der Einhaltung hoher Code-Qualitätsstandards.

12.2 Wichtigste Projektergebnisse

- **Funktionsfähige Webanwendung:** Vollständige Implementierung der Kern-User-Stories (Frontend-SPA und Backend-REST-API).
- **Robuste Architektur:** Modulare Bausteinstruktur mit klar definierten Schnittstellen, State-Management und Validierung.
- **Umfassende Dokumentation:** Vollständige Architekturdokumentation (inkl. ADRs), lückenloses Arbeitsjournal und diese Reflexion.
- **Automatisierung & Tooling:** Schnelle Build- und Test-Workflows mit Typst-Dokumentationspipeline.

13 Zielerreichung & Soll-Ist-Vergleich

13.1 Funktionaler Soll-Ist-Vergleich

Die nachfolgende Tabelle vergleicht die zu Beginn definierten Anforderungen mit dem tatsächlich erreichten Umsetzungsstand:

Anforderung	Soll-Beschreibung	Status	Ist-Zustand / Bemerkung
F01 - Auth	Registrierung, Login, JWT-Generierung	Erfüllt	Vollständig implementiert inkl. Validierung und Secure Token Handling.
F02 - CRUD	Erstellen, Bearbeiten, Löschen und Anzeigen der Kernobjekte	Erfüllt	Vollständig mit Bestätigungsdialogen und Optimistic UI Updates.
F03 - Filter/Suche	Dynamische Filterung & Volltextsuche im Frontend	Erfüllt	Debounced Search mit client-seitigem Caching.
F04 - Responsive UI	Nutzbarkeit auf Desktop, Tablet und Smartphone	Erfüllt	Responsive Breakpoints getestet auf gängigen Bildschirmgrößen.

Anforderung	Soll-Beschreibung	Status	Ist-Zustand / Bemerkung
F05 - Zusatzfeature	Erweitertes Feature (z. B. Export, Dark Mode, Statistiken)	Teilweise	Grundversion vorhanden, weitere Visualisierungen als Ausblick.

13.2 Nicht-funktionale Anforderungen

- **Performance:** Ladezeiten liegen deutlich unter den anvisierten Grenzwerten. Initial Load < 1.2s.
- **Code-Qualität:** TypeScript Strict Mode ohne Typfehler, saubere Formatierung über Linter.
- **Testabdeckung:** Wichtigste Business-Services und Controller sind mit Unit-Tests abgedeckt.

14 Erkenntnisse & Lessons Learned

14.1 Technische Erkenntnisse

Im Verlauf des Projekts wurden wertvolle technische Erfahrungen gesammelt:

- **TypeScript im Fullstack-Umfeld:** Die gemeinsame Nutzung von Interfaces/DTOs zwischen Frontend und Backend reduzierte Schnittstellenfehler massiv und beschleunigte die Entwicklung.
- **Zustandsverwaltung (State Management):** Ein klares unidirektionales Datenfluss-Muster verhinderte Race Conditions und Inkonsistenzen bei asynchronen API-Aufrufen.
- **Effiziente Toolchains:** Moderne Build-Tools (z. B. Vite / Angular CLI / Typst) verkürzten Feedback-Schleifen im Vergleich zu älteren Setups erheblich.

14.2 Aufgetretene Herausforderungen & Lösungen

Herausforderung	Problemursache	Ergriffene Lösung
CORS & Auth-Cookies	Unterschiedliche Ports bei lokaler Entwicklung für Frontend (4200/5173) und Backend (3000).	Einrichtung eines Reverse Proxys in der Dev-Konfiguration sowie korrekte <code>Access-Control-Allow-Credentials</code> Header.
Asynchrones State-Handling	UI flackerte bei schnellen aufeinanderfolgenden Filter-Eingaben.	Einsatz von Debouncing (300ms) und Abbrechen noch laufender HTTP-Requests (AbortController).
Komplexe Datenbankabfragen	N+1 Abfrageproblem bei verschachtelten Entitäten.	Optimierung der ORM-Abfragen via Eager-Loading bzw. gezielte <code>JOIN</code> -Statements.

15 Persönliche & Methodische Reflexion

15.1 Vorgehensweise & Arbeitsmethodik



Reflektieren Sie hier über Ihre Arbeitsweise: Wie gut funktionierte die Planung, das Zeitmanagement und die Priorisierung?

Die Entwicklung folgte einem iterativen, agilen Ansatz mit kurzen Entwicklungszyklen. Dies ermöglichte ein frühzeitiges Testen von Prototypen und ein schnelles Reagieren auf unvorhergesehene technische Hürden.

- **Was funktionierte sehr gut:**

- Frühes Festlegen der Datenstrukturen und API-Verträge.
- Kontinuierliches Führen des Arbeitsjournals direkt nach jeder Arbeitssitzung.
- Konsequente Nutzung von Feature-Branched und aussagekräftigen Commit-Messages.

- **Verbesserungspotenzial für künftige Projekte:**

- Noch detailliertere Zeitschätzungen für komplexe UI-Animationen und Edge-Case-Fehlerbehandlung einplanen.
- Automatisierte End-to-End-Tests (z. B. Playwright / Cypress) noch früher im Projektverlauf etablieren.

15.2 Kompetenzgewinn

Durch die intensive praktische Arbeit im WebLab konnten wesentliche Kompetenzen vertieft werden:

1. Konzeption und Implementierung skalierbarer Webarchitekturen.
2. Praktische Anwendung von Sicherheitsstandards im Web (OWASP, JWT, CORS).
3. Professionelle Dokumentation technischer Systeme mit Typst.

16 Ausblick & Weiterentwicklung

16.1 Potenzielle Erweiterungen

Für eine zukünftige Weiterentwicklung der Webanwendung bieten sich folgende Ausbaustufen an:

1. **Echtzeit-Synchronisation (WebSockets / SSE):** Live-Aktualisierungen bei gleichzeitiger Bearbeitung durch mehrere Benutzer.
2. **Progressive Web App (PWA) Support:** Offline-Fähigkeit durch Service Worker und Caching-Strategien.
3. **Erweiterte Analytics & Dashboard:** Visualisierung von Nutzungsstatistiken mit Diagrammen (z. B. Chart.js oder D3).
4. **Mehrsprachigkeit (i18n):** Internationalisierung der Benutzeroberfläche in Deutsch, Englisch und Französisch.

16.2 Fazit

Das Projekt WebLab konnte erfolgreich abgeschlossen werden. Die gesteckten funktionalen und nicht-funktionalen Anforderungen wurden erfüllt und die entwickelte Architektur bietet eine solide, zukunftsfähige Basis für künftige Erweiterungen.

17 TEIL III: Arbeitsjournal

18 Übersicht & Zeitbudget

18.1 Zielsetzung des Arbeitsjournals

Das Arbeitsjournal dient der kontinuierlichen Erfassung aller im Rahmen des Moduls WebLab erbrachten Arbeitsleistungen, Meilensteine, Herausforderungen und Zwischenentscheidungen. Es stellt die Nachvollziehbarkeit des Entwicklungsprozesses sicher.

18.2 Zeitbudget-Übersicht (Soll / Ist)

Das Modul umfasst ein Richtpensum von insgesamt ca. 90 bis 120 Arbeitsstunden (bzw. gemäss Modulvorgabe ECTS).

Phase	Arbeitspaket / Themenbereich	Soll (h)	Ist (h)	Status
1	Initialisierung, Recherche & Setup	15.0	14.5	Abgeschlossen
2	Architektur, Datenmodell & API-Design	20.0	19.0	Abgeschlossen
3	Backend-Entwicklung & Authentifizierung	25.0	26.0	Abgeschlossen
4	Frontend-Entwicklung (UI & State-Management)	30.0	32.5	Abgeschlossen
5	Testing, Bugfixing & Optimierung	15.0	13.0	Abgeschlossen
6	Dokumentation (Architektur, Journal, Reflexion)	15.0	14.0	Abgeschlossen
Total	Gesamtaufwand	120.0	119.0	100%

19 Chronologische Journaleinträge

i Hinweis

Fügen Sie für jede Arbeitssitzung oder Woche einen neuen `#journal-entry(...)` Block hinzu.

17.02.2026 Projekt-Kickoff & Toolchain-Einrichtung

Phase 1: Setup

3.5 h

Durchgeführte Tätigkeiten:

- Erstellung des Git-Repositories und Definition der Branching-Strategie.
- Aufsetzen des Basis-Projekts mit modernem Framework & TypeScript.
- Initialisierung der Typst-Dokumentationsvorlagen.

Erkenntnisse & Herausforderungen:

Typst erwies sich als deutlich schneller und wartungsfreundlicher im Vergleich zu bisherigen LaTeX-Setups.

Nächste Schritte:

Ausarbeitung des Architektur-Grobkonzepts und Definition des Datenmodells.

22.02.2026 Konzeption des Datenmodells und REST-API-Schnittstellen**Phase 2: Architektur****4.0 h****Durchgeführte Tätigkeiten:**

- Entwurf des ER-Diagramms und Spezifikation der Entitäten (User, Items).
- Definition der REST-Endpunkte (OpenAPI / Swagger Entwurf).
- Erstellung von ADR-001 (Framework) und ADR-002 (JWT-Auth).

Erkenntnisse & Herausforderungen:

Frühzeitige Einigung auf genaue DTO-Typen spart später bei der Integration viel Zeit.

Nächste Schritte:

Aufsetzen des Node.js/Express Backend-Skeletts mit ORM.

01.03.2026 Implementierung der Backend-Kernservices & Auth**Phase 3: Backend****5.5 h****Durchgeführte Tätigkeiten:**

- Aufbau der Datenbankmigrationen und Seed-Daten.
- Implementierung des Auth-Controllers (Registrierung, Login, JWT-Signierung).
- Erstellung von Middleware für Request-Validierung und geschützte Routen.

Erkenntnisse & Herausforderungen:

Passwort-Hashing mit Argon2id ist Bcrypt vorzuziehen; Integration klappte reibungslos.

Nächste Schritte:

Frontend-Authentifizierungs-Views und State-Store implementieren.

08.03.2026 Frontend-Grundgerüst, Routing und Login-Flow**Phase 4: Frontend****6.0 h****Durchgeführte Tätigkeiten:**

- Erstellung des App-Layouts mit responsivem Navigation-Header.
- Reaktive Formulare für Login und Registrierung inkl. Client-Validierung.
- Integration des Auth-States mit Token-Storage und Auth-Guards im Router.

Erkenntnisse & Herausforderungen:

Klare Trennung zwischen Presentational Components und Smart Container Components vereinfacht das Refactoring.

Nächste Schritte:

CRUD-Funktionalitäten für fachliche Hauptentitäten im Frontend.

15.03.2026**Testing, Bugfixing und Dokumentationsfinalisierung****4.5 h****Phase 5: Testing & Docs****Durchgeführte Tätigkeiten:**

- Erstellung von Unit-Tests für Auth-Service und Backend-Controller.
- Behebung von UI-Glitches auf mobilen Bildschirmauflösungen.
- Vervollständigung der Architekturdokumentation und Fazit.

Erkenntnisse & Herausforderungen:

Automatisierte Tests deckten zwei Randfall-Validierungsfehler auf.

Nächste Schritte:

Finale Abgabe und Präsentationsvorbereitung.

20 Aufwandsanalyse & Fazit zum Arbeitsprozess

20.1 Verteilung des Aufwands

Die prozentuale Aufteilung der erbrachten Stunden zeigt einen ausgewogenen Verlauf mit klarem Schwerpunkt auf der praktischen Fullstack-Implementierung:

- **Konzeption, Architektur & Setup:** ca. 28 %
- **Backend & Datenbank:** ca. 22 %
- **Frontend & UI/UX:** ca. 27 %
- **Testing, Qualitätssicherung & Dokumentation:** ca. 23 %

20.2 Erkenntnisse aus dem Arbeitsprozess

- Das Führen des Arbeitsjournals direkt im Anschluss an die Arbeitseinheiten reduzierte den Dokumentationsaufwand zum Projektende erheblich.
- Durch die modulare Struktur in Typst konnten Änderungen parallel und ohne Formatierungsbrüche gepflegt werden.